THE INVENTION, PRECISELY STATED

Deterrence by Design

An agentless method that neutralizes a database attack by turning the attacker's own request against the attacker — while it is still running.

THE ONE SENTENCE

> ***Every prior database defense ends the attacker's request as fast as possible. This invention deliberately keeps it running — replacing the attacker's in-flight SQL statement with a crafted, still-executable statement that forces the attacker's own client to wait and to hold memory — so that the act of neutralization becomes a cost imposed on the attacker, and thereby a deterrent.***

Why that is new

Conventional database security rests on four pillars — **prevent, detect, respond, recover**. Every one of them treats the attacker's request as something to stop: block it, sanitize it into an error, kill the session, or restore from backup afterward. The common reflex is to end the request as quickly as possible.

This invention adds a fifth pillar — **deterrence** — and it is not a slogan. It is a different technical operation acting on a different target. Prevention and detection act on *the request*. Deterrence, as claimed here, acts on *the attacker's own client process*: the system substitutes a crafted statement whose execution makes the attacker's session stall and consume its own memory. The measurable effect lands on the machine at the far end of the connection.

The four elements — new in combination

#	Element	What it means
1	**Agentless**	Nothing is installed on the protected server. The system acts from a separate monitoring server, through the database's own query path.
2	**In-flight**	It alters the client's SQL statement while that statement is executing, inside the live session — not before, and not after.

#	Element	What it means
3	**Dynamic**	The replacement statement is composed in dependence on the identified activity — not a fixed sanitization applied uniformly.
4	**Sustained cost**	The crafted statement remains executable; its execution delays the client and holds the client's memory. The attacker pays in time and resources.

No single element alone is the invention. Their combination is — and, as shown next, the prior art cannot be assembled to reach it.

Why nothing in the prior art reaches it

The decisive point is stronger than "the references don't show element 4." Their mechanisms are **incompatible** with it.

Reference	What it does	Why it cannot reach the invention
Alberstein (Verizon)	Converts suspect elements into non-executable identifiers that "would result in an error when executed."	An erroring statement terminates and releases its resources. It structurally cannot delay the client or hold its memory. The opposite mechanism.
Gupta (Virsec)	Detects injection via a "golden table," then terminates the session.	Requires instrumented code installed on the web/application server — the very agent element 1 forbids — and kills the session, the fast-exit this invention inverts.
Miller (Pure Storage)	Detects threats from storage I/O patterns; responds with snapshots and redundancy.	The wrong layer entirely: it never touches a database query or a live client connection.

So the invention is not "a better detector." It is a different operation on a different target: not the request — block, sanitize, alert — but the attacker's own client process, made to stall and consume itself, from outside the protected server, tailored to the attack in flight.

The proof: measured, not merely argued

The claimed method and the closest prior-art technique were run against a live Microsoft SQL Server 2022 instance, each five times. The result is a difference of kind, not degree:

Approach	Client held for	Client memory held	Outcome
Prior-art neutralization (statement errors on execution)	0.001 s	0 MB	errors instantly, frees resources
The claimed method (crafted, executable statement)	**8.44 s**	**353 MB**	executes; client stalls and holds memory

The prior art releases everything in a millisecond. The claimed method holds the attacker's client for more than eight seconds and 353 megabytes. That gap is the invention, made physical — and it is exactly the effect an error-producing sanitizer cannot produce.

Why this answers each patentability question

Not an abstract idea. A method whose output is a measured physical change in another machine's operating state — 8.4 seconds and 353 MB held — is not a mental process and is not "merely software." A person cannot rewrite a statement inside a live, executing database session.

Not anticipated. The closest reference neutralizes by making the statement error out; that mechanism cannot produce the claimed executable-and-sustained effect. The reference does not disclose the limitation — it contradicts it.

Not obvious. The references cannot be combined toward this claim: one requires the agent the claim forbids and teaches away by killing sessions; another is a non-analogous tool from an unrelated field.

In five words: the prior art says "stop the attack." This invention says "make the attacker pay."

Reference characterizations are drawn from the published texts of US 2022/0272121 (Verizon), US 2016/0337400 (Virsec Systems) and US 2021/0216666 (Pure Storage). Measurements were produced on 15 August 2026 against a Microsoft SQL Server 2022 test instance; the experiment is reproducible and the measurement program is available as an exhibit. Prepared as an explanatory summary; it is not a legal opinion and does not replace the claims, which define the invention.